Diffusion models and ODE generation

A simple flow-matching example with four Gaussian components

Directed Reading Program
Florida State University

Data Velocity Training ODE sampling

Our goal: generate a mixture from Gaussian noise

$$X(0) \sim \mathcal{N}(0, I_2)$$

$$X(1) \text{ approximately follows } \frac{1}{4}\sum_{j=1}^{4}\mathcal{N}(\mu_j, 0.35^2 I_2)$$

- One sample is a point with two coordinates.
- We learn a velocity field that moves a cloud of points.
- A fresh Gaussian starting point produces a new generated sample.

What is a four-component Gaussian mixture?

$$\mu_1 = (-2, -2), \quad \mu_2 = (-2, 2), \quad \mu_3 = (2, -2), \quad \mu_4 = (2, 2)$$

1. Choose one center with probability $1/4$.
2. Draw $\varepsilon \sim \mathcal{N}(0, I_2)$.
3. Return $x = \mu_j + 0.35\varepsilon$.

The covariance is $0.35^2 I_2$. Each cluster has width, not just a single point.

Direct sampling already works. We use the known mixture to teach and check a learned generator.

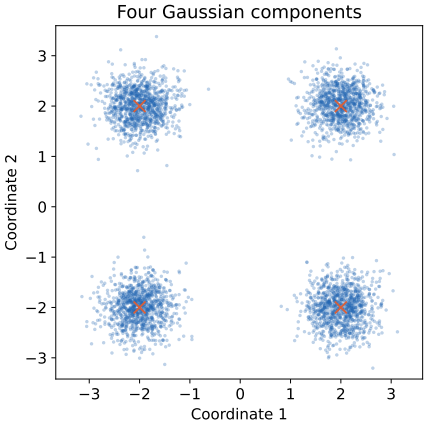
1. Create the training data

```
centers = torch.tensor([
    [-2., -2.], [-2., 2.],
    [2., -2.], [2., 2.]
])
sigma = 0.35

def sample_data(n):
    labels = torch.randint(4, (n,))
    return (centers[labels]
            + sigma * torch.randn(n, 2))

x_train = sample_data(4096)
```

Shape: `(4096, 2)`. The network receives samples, not their component labels.



An ODE specifies how a point moves

$$\boxed{\frac{dX(t)}{dt} = v_\theta(X(t), t)}$$

- $X(t)$ is the current position in the plane.
- $v_\theta(x, t)$ is a two-dimensional velocity.
- θ contains the network's weights and biases.

Our convention: $t = 0$ means noise and $t = 1$ means data.

This is generation time, not physical time or a time-series index.

One Euler step

$$X_{k+1} = X_k + \Delta t \, v_\theta(X_k, t_k)$$

An illustrative calculation:

$$X_k = (0, 1), \qquad v_\theta(X_k, t_k) = (2, -1), \qquad \Delta t = 0.1$$

$$X_{k+1} = (0, 1) + 0.1(2, -1) = (0.2, 0.9).$$

We approximate a short motion using the velocity at the start of the step.

At the next step, evaluate the velocity at the new position and time.

Diffusion and flow matching: related approaches

	Score-based diffusion	This notebook
Training construction	Add noise to data	Interpolate noise and data
Learned quantity	Score $\nabla_x \log p_t(x)$	Velocity $v_\theta(x, t)$
Generation	Reverse SDE or probability-flow ODE	Forward ODE from noise to data

We implement flow matching, not a DDPM or Brownian-motion simulation.

The ODE makes the transport idea accessible with basic calculus.

Lipman et al., Flow Matching for Generative Modeling; Liu et al., Rectified Flow.

2. Construct straight training paths

Choose independently:

$$x_0 \sim \mathcal{N}(0, I_2), \qquad x_1 \sim \text{training data}, \qquad t \sim U[0,1].$$

$$\boxed{x_t = (1-t)x_0 + t x_1}$$

$t = 0$	$t = 1/2$	$t = 1$
x_0	$(x_0 + x_1)/2$	x_1

We can sample a point on this path directly, without solving an ODE.

Lipman et al., Flow Matching for Generative Modeling; Liu et al., Rectified Flow.

The target velocity is easy to calculate

$$\frac{d}{dt}\Big((1-t)x_0 + tx_1\Big) = x_1 - x_0$$

$$\boxed{u = x_1 - x_0}$$

Example: $x_0 = (0, 0)$ and $x_1 = (2, 2)$.

At $t = 0.25$, the position is $x_t = (0.5, 0.5)$ and the target velocity is $(2, 2)$.

The position changes along this training segment. Its target velocity stays constant.

What does the network learn from many pairs?

Different noise-data pairs can give the same (x_t, t) with different velocities.

$$v^*(x, t) = \mathbb{E}[X_1 - X_0 \mid X_t = x]$$

Squared-error regression learns the conditional average velocity.

Under suitable regularity assumptions, the ideal field transports the interpolation distributions.

The learned ODE paths need not be straight or end at a paired training sample.

Lipman et al., Flow Matching for Generative Modeling; Liu et al., Rectified Flow.

3. A network for position and time

```
model = nn.Sequential(
    nn.Linear(3, 64),
    nn.SiLU(),
    nn.Linear(64, 64),
    nn.SiLU(),
    nn.Linear(64, 2)
)
```

Input: $(x^{(1)}, x^{(2)}, t)$

Output: $(v^{(1)}, v^{(2)})$

Two hidden layers of width 64.

4,546 trainable parameters.

Smooth activation: $\mathrm{SiLU}(a) = a\sigma(a)$.

Each training row has its own time

```
t = torch.rand(256, 1)
xt = (1 - t) * x0 + t * x1
prediction = model(torch.cat([xt, t], dim=1))
```

Tensor	Shape
x0, x1, xt	256×2
t	256×1
Network input	256×3
Predicted velocity	256×2

The same location can require different motion at different generation times.

4. Mean squared velocity error

$$L(\theta) = \frac{1}{2B} \sum_{i=1}^{B} \| v_\theta(x_{t_i}, t_i) - (x_{1,i} - x_{0,i}) \|^2$$

```
loss_fn = nn.MSELoss()
```

PyTorch averages over all B examples and both output coordinates.

We compare velocities, not generated endpoints.

The target is a vector with two entries, not a real/fake label.

One optimizer updates the velocity network

```
optimizer = torch.optim.Adam(
    model.parameters(), lr=0.001
)
```

- The optimizer adjusts the same weights and biases across all times.
- Adam uses running averages of gradients and squared gradients.
- We use its default averaging settings and learning rate 0.001.

There is one regression objective and no discriminator.

5. Training: create a batch and predict velocity

```
history = []
for step in range(5000):
    x1 = x_train[torch.randint(len(x_train), (256,))]
    x0 = torch.randn(256, 2)
    t = torch.rand(256, 1)
    xt = (1 - t) * x0 + t * x1
    target = x1 - x0
    prediction = model(torch.cat([xt, t], dim=1))
    loss = loss_fn(prediction, target)
```

Fresh noise and times at each iteration. Real examples sample the training set with replacement.

The network sees only `xt` and `t`.

Training: update the parameters

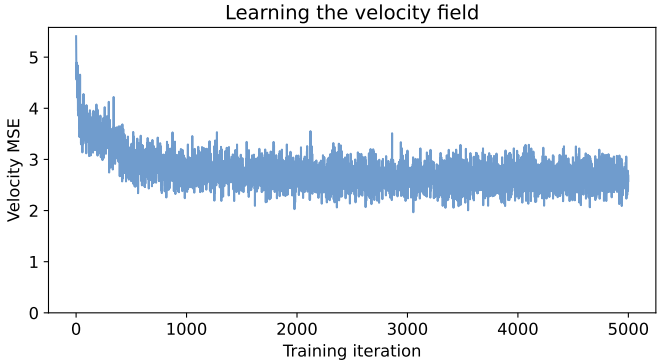
Continuation inside the same loop:

```
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    history.append(loss.item())
```

`zero_grad()`	Clear previous gradients.
`backward()`	Differentiate the velocity loss.
`step()`	Update the network parameters.

Training does not require an ODE solve. Sampling will.

Why the loss does not approach zero



The target velocity varies across pairs, even at the same position and time.

The best predictor averages these targets. Their conditional variance leaves residual error.

6. ODE sampling with forward Euler

```
def generate(z, steps=100):
    x = z.clone()
    path = [x.clone()]
    with torch.no_grad():
        for k in range(steps):
            t = torch.full((len(x), 1), k / steps)
            x = x + model(torch.cat([x, t], dim=1)) / steps
            path.append(x.clone())
    return x, torch.stack(path)
```

$\Delta t = 1/\texttt{steps}$. The plus sign advances from noise to data.

The model stays fixed. We record positions only to visualize the trajectories.

A fresh Gaussian input gives a new sample

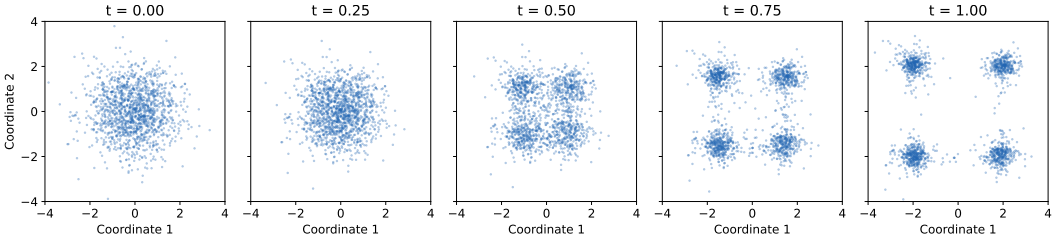
```
torch.manual_seed(2026)
model.eval()
z = torch.randn(5000, 2)
real_test = sample_data(5000)
generated, path = generate(z)
```

The sampler uses Gaussian inputs, time, and the learned velocity network.

`real_test` is only for evaluation; it never enters `generate`.

The same input and fixed model give the same numerical trajectory.

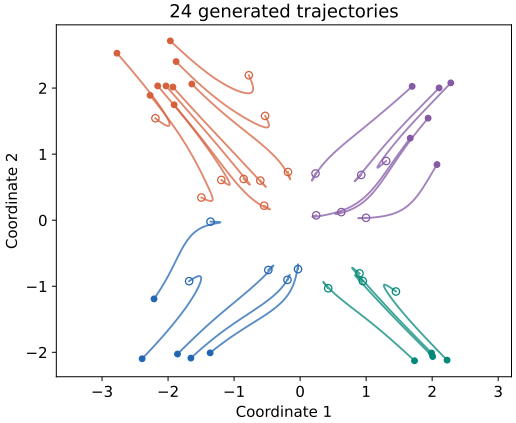
The cloud evolves into four clusters



The panels follow the same 1,500 generated particles.

No new noise enters between Euler steps. Randomness comes from the starting points.

Individual generated trajectories



24 trajectories from the learned ODE.

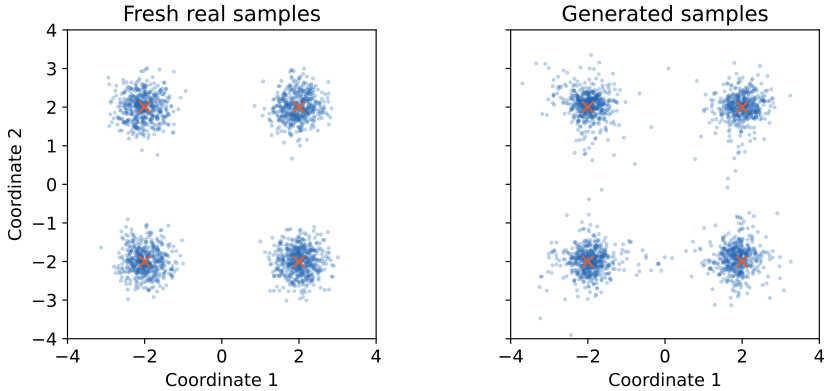
Open circles: Gaussian starts.

Filled dots: generated endpoints.

Colors identify the nearest final center.

These paths can curve, although the training interpolations were straight.

7. Fresh real and generated samples



Both panels use equal coordinate scales and display 2,000 points.

We compare distributions, not a generated point with a paired correct endpoint.

Checking all four clusters and their spread

```
distance, group = torch.cdist(points, centers).min(dim=1)
shares = 100 * torch.bincount(group, minlength=4) / len(points)
radius = distance.square().mean().sqrt()
nearby = 100 * (distance < 1).float().mean()
```

- Mode shares: does each center receive roughly 25% of samples?
- RMS distance: how spread out are samples around their nearest centers?
- Within radius 1: how much mass lies near the clusters?

Nearest-center assignments are diagnostics, not inferred true component labels.

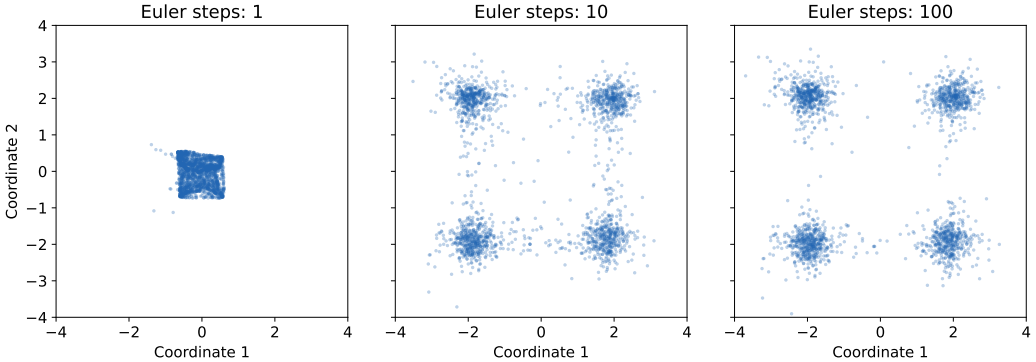
Results on 5,000 new samples

Diagnostic	Fresh real	Generated
Nearest $(-2,-2)$ (%)	24.9000	25.9800
Nearest $(-2,2)$ (%)	24.3800	25.8400
Nearest $(2,-2)$ (%)	25.8600	23.9600
Nearest $(2,2)$ (%)	24.8600	24.2200
RMS distance to nearest center	0.4934	0.5187
Within distance 1 of a center (%)	98.4400	95.2000

Finite-sample proportions differ from the theoretical 25% per component.

Good mode counts alone are insufficient. Four collapsed points could have the same counts.

An experiment: 1, 10, or 100 Euler steps



Same model and same initial noise in all three panels. No retraining.

For the zero-mean target, $v^*(x, 0) = -x$: one unit Euler step gives $x - x = 0$.

Smaller steps reduce numerical error; they cannot repair a poorly learned field.

Diffusion models also have an ODE sampler

For a forward diffusion with scalar, state-independent noise coefficient:

$$dX = f(X, \tau)\, d\tau + g(\tau)\, dW_\tau$$

$$\frac{dX}{d\tau} = f(X, \tau) - \frac{1}{2} g(\tau)^2 \nabla_x \log p_\tau(X)$$

W_τ supplies Brownian noise; $\nabla_x \log p_\tau$ is the score.

With the exact score and regularity assumptions, the SDE and probability-flow ODE have the same one-time distributions, not the same paths.

Diffusion generation runs this ODE backward from noise to data. Our notebook directly learns a velocity with t increasing from noise to data.

Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, Section 4.3.

What the example establishes

- A velocity network and an ODE solver can turn Gaussian inputs into a multimodal sample distribution.
- Finite data, approximation, optimization, and the solver all contribute error.
- Generation time is an artificial parameter. These two-dimensional outputs are not time series.
- A future model could generate an entire time series and evaluate temporal structure using suitable tools, potentially including Volterra signatures.

Experiments and further reading

1. Change the seed and compare cluster coverage and spread.
2. Train for 1,000 iterations, keeping 100 sampling steps.
3. Make the target clusters narrower by reducing `sigma`.
4. Remove the time input from both training and sampling. What changes?

Restart the kernel for each experiment. Use fresh test samples after tuning.

References: Lipman et al., Flow Matching; Liu et al., Rectified Flow; Song et al., Score-Based Generative Modeling.

Companion notebook: `learning_flow_matching.ipynb`